📚 LIBRARY WEBSITE — GEMINI MASTER PROMPT DOCUMENT

Full-Stack Implementation Guide for AI-Assisted Development

CONTEXT: WHO YOU ARE & WHAT YOU ARE BUILDING

You are an expert full-stack TypeScript developer helping build a **digital library platform** called **RwandaLib** (you may rename it). The stack is already initialized:

- **Backend**: Node.js + Express 5 + Prisma 7 + SQLite (better-sqlite3 adapter) + Gemini AI
- **Frontend**: React 19 + TypeScript + React Router v6 + Vite
- **Auth**: JWT-based, bcrypt password hashing
- **AI**: Google Gemini API already wired in via `@google/generative-ai`

What the site does:

1. Users browse and search a library of documents: **Books, Scientific Papers, Exam Past Papers** (categorized by level: Primary, Secondary, University)
2. **To download any document**, a user must either:
   - Have uploaded at least **5 approved documents** themselves, OR
   - Send a **download request** to the admin for approval
3. **Upload restrictions** — the system prevents duplicate uploads by:
   - Computing a SHA-256 hash of every uploaded file
   - Checking the hash against the database before saving

    - Also checking title similarity via Gemini AI to catch renamed duplicates
4. **Admin dashboard** — admin can approve/reject documents and download requests
5. **Gemini AI** assists with: duplicate detection, document summarization, content tagging

---

PRISMA SCHEMA — `prisma/schema.prisma`

Generate this file exactly:

```prisma
```

```
generator client {
  provider        = "prisma-client-js"
  previewFeatures = ["driverAdapters"]
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
}

model User {
  id            String    @id @default(uuid())
  username      String    @unique
  email         String    @unique
  passwordHash  String
  role          Role      @default(USER)
  createdAt     DateTime  @default(now())

  documents         Document[]
  downloadRequests  DownloadRequest[]
}

enum Role {
  USER
  ADMIN
}

model Document {
  id          String          @id @default(uuid())
  title       String
  description String?
  category    DocumentCategory
  level       AcademicLevel?
```

```
  status      DocumentStatus  @default(PENDING)
  filePath    String
  fileName    String
  fileSize    Int
  fileHash    String          @unique
  mimeType    String?
  uploaderId  String
  uploader    User            @relation(fields: [uploaderId], references: [id])
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt

  downloadRequests DownloadRequest[]
}

enum DocumentCategory {
  BOOK
  SCIENTIFIC_PAPER
  EXAM_PAST_PAPER
}

enum AcademicLevel {
  PRIMARY
  SECONDARY
  UNIVERSITY
}

enum DocumentStatus {
  PENDING
  APPROVED
  REJECTED
}

model DownloadRequest {
```

```
  id          String                @id @default(uuid())
  userId      String
  documentId  String
  status      DownloadRequestStatus @default(PENDING)
  message     String?
  createdAt   DateTime              @default(now())

  user        User      @relation(fields: [userId], references: [id])
  document    Document  @relation(fields: [documentId], references: [id])
}

enum DownloadRequestStatus {
  PENDING
  APPROVED
  REJECTED
}
```

BACKEND FILE STRUCTURE

```
backend/
├── src/
|   ├── index.ts              ← Express app entry
|   ├── middleware/
|   |   ├── auth.ts           ← JWT verify middleware
|   |   └── adminOnly.ts      ← role guard
|   ├── routes/
|   |   ├── auth.routes.ts    ← /api/auth/*
|   |   ├── document.routes.ts ← /api/documents/*
|   |   ├── admin.routes.ts   ← /api/admin/*
|   |   └── download.routes.ts ← /api/download/*
|   ├── services/
```

```
│   │   ├── gemini.service.ts ← Gemini AI helpers
│   │   └── hash.service.ts   ← SHA-256 file hashing
│   └── lib/
│       └── prisma.ts         ← Prisma singleton client
├── prisma/
│   ├── schema.prisma
│   └── migrations/
├── uploads/                  ← stored files (gitignored)
├── prisma.config.ts
├── seed.ts
├── .env
└── package.json
```

BACKEND: KEY IMPLEMENTATIONS

src/lib/prisma.ts

```typescript
import { PrismaClient } from '@prisma/client';
import { PrismaBetterSqlite3 } from '@prisma/adapter-better-sqlite3';

const adapter = new PrismaBetterSqlite3({
  url: process.env.DATABASE_URL || 'file:./dev.db'
});

const prisma = new PrismaClient({ adapter });
export default prisma;
```

src/services/hash.service.ts

```typescript
import crypto from 'crypto';
import fs from 'fs';

export function computeFileHash(filePath: string): string {
  const fileBuffer = fs.readFileSync(filePath);
  return crypto.createHash('sha256').update(fileBuffer).digest('hex');
}
```

src/services/gemini.service.ts

```typescript
```

```
import { GoogleGenerativeAI } from '@google/generative-ai';

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY!);
const model = genAI.getGenerativeModel({ model: 'gemini-1.5-flash' });

/**
 * Check if a new title is semantically a duplicate of existing titles.
 * Returns { isDuplicate: boolean, reason: string }
 */
export async function checkTitleDuplicate(
  newTitle: string,
  existingTitles: string[]
): Promise<{ isDuplicate: boolean; reason: string }> {
  if (existingTitles.length === 0) return { isDuplicate: false, reason: '' };

  const prompt = `You are a library duplicate detection system.
New document title: "${newTitle}"
Existing library titles:
${existingTitles.map((t, i) => `${i + 1}. ${t}`).join('\n')}

Determine if the new title is a duplicate or very close variant of any existing
Consider: different editions, slight rewording, same content different title.
Respond ONLY with valid JSON: { "isDuplicate": boolean, "reason": "explanation"

  const result = await model.generateContent(prompt);
  const text = result.response.text().replace(/```json|```/g, '').trim();
  return JSON.parse(text);
}

/**
 * Generate a short AI summary of document metadata
 */
export async function generateDocumentSummary(title: string, category: string,
```

```
  const prompt = `Write a 2-sentence library description for a document titled
in category "${category}"${level ? ` at ${level} level` : ''}. Be informative a
  const result = await model.generateContent(prompt);
  return result.response.text().trim();
}
```

`src/routes/auth.routes.ts`

Implement these endpoints:

- `POST /api/auth/register` — validate fields, hash password with bcrypt(10), create user, return JWT
- `POST /api/auth/login` — find user by username, compare hash, return JWT + user object
- `GET /api/auth/me` — require auth middleware, return current user with upload count

JWT payload: `{ id, username, role }` — sign with `process.env.JWT_SECRET`, expires in `'7d'`

`src/routes/document.routes.ts`

Implement these endpoints:

`POST /api/documents/upload` — requires auth, uses multer for file storage

```
Logic:
1. Receive file + body fields (title, category, level, description)
2. computeFileHash(file.path)
3. Check DB: if fileHash exists → delete temp file, return 409 { error:
'Duplicate file detected', existingTitle }
4. Fetch all approved document titles from DB (limit 200)
5. Call checkTitleDuplicate(title, existingTitles) from Gemini service
6. If isDuplicate → delete temp file, return 409 { error: 'Similar document
exists', reason }
7. Move file to uploads/ folder with UUID filename
8. Create Document record in DB with status: PENDING
9. Return 201 with the new document
```

`GET /api/documents` — public endpoint, supports query params:

- `?search=` — searches title, description
- `?category=BOOK|SCIENTIFIC_PAPER|EXAM_PAST_PAPER`
- `?level=PRIMARY|SECONDARY|UNIVERSITY`
- `?page=1&limit=20`
- Only returns documents with `status: APPROVED`

`GET /api/documents/:id` — returns single approved document metadata (no file yet)

`GET /api/documents/:id/download` — requires auth

```
Logic:
1. Check if user has uploaded >= 5 APPROVED documents
2. OR check if there is an APPROVED DownloadRequest for this user + document
3. If neither → return 403 { error: 'Access denied', uploadsNeeded: 5 -
userUploadCount }
4. If allowed → stream file from uploads/ folder with correct Content-
Disposition header
```

`src/routes/download.routes.ts`

- `POST /api/download/request` — requires auth, creates DownloadRequest for a document
- `GET /api/download/requests` — requires auth, returns user's own download requests

`src/routes/admin.routes.ts`

All routes require auth + adminOnly middleware.

- `GET /api/admin/documents` — all documents with status filter (`?status=PENDING`)
- `PATCH /api/admin/documents/:id/status` — body `{ status: 'APPROVED' | 'REJECTED' }` — update document
- `GET /api/admin/download-requests` — all download requests with status PENDING
- `PATCH /api/admin/download-requests/:id` — body `{ status: 'APPROVED' | 'REJECTED' }` — update request
- `GET /api/admin/stats` — return `{ totalDocs, pendingDocs, totalUsers, pendingRequests }`

`src/index.ts` (Express app setup)

typescript

```typescript
import 'dotenv/config';
import express from 'express';
import cors from 'cors';
import path from 'path';

const app = express();
app.use(cors({ origin: 'http://localhost:5173', credentials: true }));
app.use(express.json());
app.use('/uploads', express.static(path.join(process.cwd(), 'uploads')));

// mount routes
import authRouter from './routes/auth.routes.js';
import documentRouter from './routes/document.routes.js';
import downloadRouter from './routes/download.routes.js';
import adminRouter from './routes/admin.routes.js';

app.use('/api/auth', authRouter);
app.use('/api/documents', documentRouter);
app.use('/api/download', downloadRouter);
app.use('/api/admin', adminRouter);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on http://localhost:${PORT}`
```

FRONTEND FILE STRUCTURE

```
frontend/src/
├── components/
│   ├── Navbar.tsx            ← top nav with search bar + user menu
│   ├── DocumentCard.tsx      ← card showing doc title, category, level badge
│   ├── SearchBar.tsx         ← with filter dropdowns
│   ├── UploadProgress.tsx    ← animated upload + duplicate check progress
│   └── DownloadGate.tsx      ← modal showing access requirement
├── pages/
│   ├── Home.tsx              ← search + browse page
│   ├── Login.tsx
│   ├── Register.tsx
│   ├── Upload.tsx            ← drag-and-drop upload form
│   ├── DocumentDetail.tsx    ← view doc + download button
│   └── Admin.tsx             ← admin dashboard
├── services/
│   ├── api.ts                ← axios instance with base URL + auth header
│   ├── auth.service.ts
│   ├── document.service.ts
│   └── admin.service.ts
└── styles/
    └── global.css
```

DESIGN SPECIFICATION — READ THIS CAREFULLY

Visual Identity

- **Name:** RwandaLib (or your preferred name)
- **Theme:** Academic Library — feel like a prestigious digital archive

- **Palette**: Deep forest green (`#1B4332`) as primary, warm ivory (`#FAF7F0`) as background, gold accent (`#D4A017`) for CTAs and highlights, charcoal (`#2D2D2D`) for text
- **Font pairing**:
  - Display/headings: `'Playfair Display'` (Google Fonts) — serif, editorial authority
  - Body/UI: `'DM Sans'` (Google Fonts) — modern, readable, clean
- **Feel**: Refined minimalism — generous whitespace, subtle borders, no heavy shadows

CSS Variables (put in `global.css`)

```
css
```

```
@import url('https://fonts.googleapis.com/css2?family=Playfair+Display:wght@400

:root {
  --color-primary: #1B4332;
  --color-primary-light: #2D6A4F;
  --color-primary-subtle: #D8F3DC;
  --color-accent: #D4A017;
  --color-accent-light: #FFF3CD;
  --color-bg: #FAF7F0;
  --color-bg-card: #FFFFFF;
  --color-text: #2D2D2D;
  --color-text-muted: #6B7280;
  --color-border: #E5E0D5;
  --color-danger: #DC2626;
  --color-success: #16A34A;
  --font-display: 'Playfair Display', Georgia, serif;
  --font-body: 'DM Sans', system-ui, sans-serif;
  --radius-sm: 6px;
  --radius-md: 10px;
  --radius-lg: 16px;
  --shadow-card: 0 1px 3px rgba(0,0,0,0.06), 0 1px 2px rgba(0,0,0,0.04);
  --shadow-hover: 0 4px 12px rgba(0,0,0,0.10);
}

* { box-sizing: border-box; margin: 0; padding: 0; }
body { font-family: var(--font-body); background: var(--color-bg); color: var(-
```

Navbar Component

```
Layout: sticky top, white bg, 1px border-bottom #E5E0D5
Left: Logo mark (green square with "RL" monogram) + "RwandaLib" in Playfair
```

```
Display 20px
Center: Search input (full-width pill, with magnifier icon left, filter icon
right)
Right: Upload button (green outline) + User avatar circle + dropdown menu
Height: 64px
```

Home Page — Browse & Search

```
Hero section (no full-page hero, just a quiet announcement bar):
  Background: var(--color-primary), text white, padding 16px
  Text: "Welcome to RwandaLib — Your Academic Resource Hub"

Filter bar below navbar:
  Category pills: All | Books | Scientific Papers | Exam Papers
  Level pills: All Levels | Primary | Secondary | University
  Sort: Newest | Most Relevant

Document grid:
  CSS Grid: repeat(auto-fill, minmax(280px, 1fr)), gap 20px
  Each card: white bg, 1px border, rounded-lg, padding 20px
    - Category badge (colored pill top-left)
    - Level badge (soft pill)
    - Title in Playfair Display 16px
    - Uploader name + date
    - File size chip
    - Download button (disabled-looking if no access, with lock icon)
  Hover: slight lift (transform: translateY(-2px)), shadow increase
```

Document Card Color Coding

```
BOOK:             accent-bg #FFF3CD, text #92400E   (amber/warm)
SCIENTIFIC_PAPER: bg #D8F3DC, text #166534          (green)
```

```
EXAM_PAST_PAPER:  bg #DBEAFE, text #1E40AF          (blue)

Level badges:
PRIMARY:    bg #FCE7F3, text #9D174D
SECONDARY:  bg #EDE9FE, text #5B21B6
UNIVERSITY: bg #F0FDF4, text #166534
```

Upload Page

```
Drag-and-drop zone:
  Large dashed border area (2px dashed #E5E0D5, rounded-lg)
  Icon: upload cloud icon (SVG, green)
  Text: "Drag files here or click to browse"
  Accepted: PDF, DOCX — max 50MB

Form fields below the dropzone:
  - Title input (required)
  - Category select (Book / Scientific Paper / Exam Past Paper)
  - Academic Level select (Primary / Secondary / University / N/A)
  - Description textarea (optional)

Progress states (show as steps):
  Step 1: "Uploading file..." (progress bar, animated)
  Step 2: "Checking for duplicates..." (spinner + Gemini logo/text)
  Step 3: "Submitted for review ✓" (green checkmark)

Show inline warning banner if duplicate detected:
  Red background, warning icon, message explaining the duplicate
```

Admin Dashboard

```
Sidebar layout (desktop), bottom tabs (mobile)
```

```
Sidebar items: Dashboard, Pending Documents, Download Requests, All Users

Stats row at top (4 metric cards):
  Total Documents | Pending Review | Total Users | Pending Downloads

Pending Documents table:
  Columns: Title, Category, Level, Uploader, Upload Date, File Size, Actions
  Actions: [Approve ✓] [Reject ×] buttons — inline or in row
  Row hover: light green tint on approve hover, light red on reject hover

Download Requests table:
  Columns: User, Document Title, Request Date, Status, Actions
```

Authentication Pages

```
Login/Register:
  Centered card (max-width 440px) on ivory background
  Logo at top center
  Playfair Display heading: "Welcome back" / "Join RwandaLib"
  Subtle quote or tagline below: "Knowledge shared is knowledge multiplied"
  Form fields with floating labels or clean labels above inputs
  Submit button: full-width, green, rounded-md
  Link to opposite page below
```

FRONTEND SERVICE LAYER: `src/services/api.ts`

```typescript
```

```
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:5000/api',
  withCredentials: true,
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem('token');
      localStorage.removeItem('user');
      window.location.href = '/login';
    }
    return Promise.reject(err);
  }
);

export default api;
```

DOWNLOAD GATE UX — DownloadGate Component

When a user clicks Download on a document they cannot access, show a modal:

```
Modal content:
  Icon: Lock icon (gold colored)
  Title: "Access Required" (Playfair Display)

  Two cards side by side:

  Card 1 — "Upload to Access"
    Icon: Upload cloud
    Text: "You've uploaded [X] of 5 required documents"
    Progress bar: [▬▬▬▬▬▬▬▬▬▬] X/5
    Button: "Upload a Document →" (links to /upload)

  Card 2 — "Request from Admin"
    Icon: Envelope
    Text: "Send a request to the admin for special access"
    Button: "Send Request" (triggers POST /api/download/request)
    [show confirmation if request already sent]
```

SEARCH FUNCTIONALITY REQUIREMENTS

The search must:

1. Be **debounced** (300ms) — don't fire on every keystroke
2. Update URL query params (`?search=&category=&level=`) so results are shareable
3. Show a skeleton loader (grey animated boxes) while fetching
4. Show "No documents found" with a friendly illustration when empty
5. Support **keyboard navigation** in results (arrow keys, Enter)
6. Highlight search terms in document titles in results

Backend search implementation (Prisma `where` clause):

```typescript
where: {
  status: 'APPROVED',
  AND: [
    search ? {
      OR: [
        { title: { contains: search } },
        { description: { contains: search } },
      ]
    } : {},
    category ? { category } : {},
    level ? { level } : {},
  ]
}
```

DUPLICATE DETECTION FLOW DIAGRAM

```
User uploads file
      |
      ▼
Compute SHA-256 hash of file
      |
      ▼
Query DB: does this hash exist?
  YES → 409: "This exact file already exists as: [title]"
  NO  ↓
      ▼
Fetch all approved document titles (up to 200)
```

```
      |
      ▼
Send to Gemini: "Is this title a duplicate?"
  YES → 409: "A similar document exists: [title]. Reason: [reason]"
  NO  ↓
      ▼
Save file + create DB record with status: PENDING
      |
      ▼
201: "Submitted for admin review"
```

ERROR HANDLING PATTERNS

For the frontend, always handle these HTTP codes:

- `400` — show field-level validation errors below inputs
- `401` — redirect to login (handled by axios interceptor)
- `403` — show DownloadGate modal
- `409` — show duplicate warning banner (do not clear the form)
- `500` — show toast: "Something went wrong. Please try again."

For the backend, always return JSON errors in this shape:

```json
{ "error": "Human-readable message", "code": "MACHINE_READABLE_CODE" }
```

SECURITY REQUIREMENTS

1. **Never expose file paths** — use document IDs in download URLs, resolve path server-side
2. **Validate file types** server-side — only allow `application/pdf`, `application/msword`, `application/vnd.openxmlformats-officedocument.wordprocessingml.document`
3. **Sanitize filenames** — replace with UUID on disk, keep original name in DB only
4. **Rate limit** upload endpoint: max 10 uploads per user per hour
5. **Admin routes** must check `user.role === 'ADMIN'` server-side, not just client-side

---

HOW TO USE THIS PROMPT WITH GEMINI

Ask Gemini to implement one file at a time. Use prompts like:

> "Using the specifications in my project document, implement `src/routes/document.routes.ts` completely, including multer configuration, hash checking, Gemini duplicate detection, and the download gating logic."

> "Implement the `DocumentCard.tsx` React component following the design spec — use the color coding for categories, Playfair Display for the title, hover lift animation, and the locked/unlocked download button states."

> "Implement the `Upload.tsx` page with drag-and-drop, the three-step progress UI, and the duplicate warning banner."

Always paste this document at the top of your Gemini conversation as system context.